



GDB debugging workshop

Session 4



Session 4 agenda

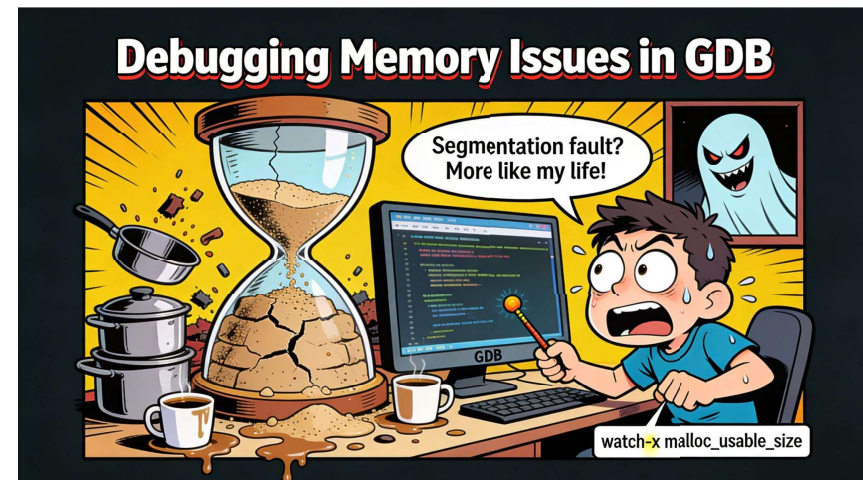
- Tracepoints - debt
- Debugging memory issues with GDB
- Runtime memory analysis with Valgrind + Valgrind - GDB integration
- ASAN and UBSAN
- Heap inspection



Tracepoints - saving to file

[day02/05_tracepoints](#)

Debugging memory issues with GDB





Common memory issues in C-and-C++-based systems

- **Memory leaks** - Allocated memory never freed, gradual resource exhaustion
- **Buffer overflows** - Accessing memory beyond allocated boundaries
- **Use-after-free** - Accessing deallocated memory, undefined behavior
- **Double-free** - Freeing same memory twice
- **Memory corruption** - Writing wrong types or to a wrong address
- **Uninitialized memory** - Reading from unallocated or uninitialized pointers



Selected GDB CLI commands for memory debug

- `x/[COUNT][ELEMENT SIZE][FMT] <address>`
 - Display memory contents in various formats
- `catch syscall <brk/mmap/...>` - Intercept heap allocation syscalls
- `info proc mappings` - View process memory layout and regions



Creating `x/[COUNT][ELEMENT SIZE][FMT]` expression

- All three modifiers are optional
- `COUNT` is how many units to display, and it must come first if present
- `ELEMENT SIZE` selects the unit width
(b = 1 byte, h = 2 bytes, w = 4 bytes, g = 8 bytes)
- `FMT` selects how each unit is printed
(x = hex, d = signed decimal, u = unsigned decimal, t = binary, c = char, s = string, i = instruction)



Some practical examples

- `x/4wx $sp`: Show 4 words from `$sp`, formatted as hex.
- `x/16bx buf`: Show 16 bytes from `buf`, formatted as hex.
- `x/3i $pc`: Disasm 3 instructions starting at the current program counter.
- `x/8tb flags`: Show 8 bytes starting at `flags`, each byte printed in binary.
- `x/s name`: Show a null-terminated single-byte string starting at `name`.



Displaying a buffer in memory - example

[day02/14_memory_dump](#)



catch syscall <brk/mmap/...>

.text (code) | .data (initialized) | .bss (uninitialized) | ← program break ← heap |
stack

"brk" originates from "break" - specifically the **program break**, which marks the boundary between a process's initialized data segment (BSS) and the start of the heap.

Relevant syscalls: brk, mmap, mremap, munmap



Checking for a dynamic memory allocations

[day02/14_memory_dump](#)

We have:

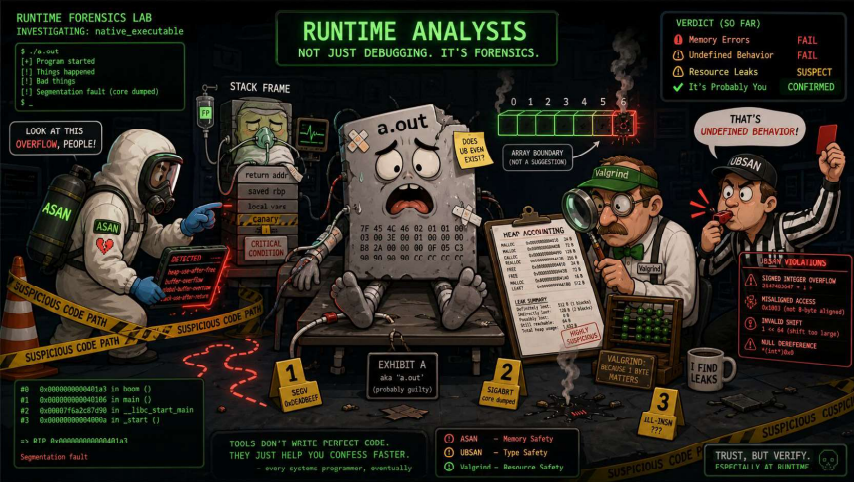
- `std::unique_ptr` creation
- Allocating a memory block using operator **new**

How many actual syscalls will be caught?



MALLOC_CHECK_ (obsolete)

- Environment variable in glibc that switches **malloc/free/realloc** to a special **debug** implementation with extra heap consistency checks.
- Set with **MALLOC_CHECK_=0/1/2/3**
- Designed to help diagnose heap corruption bugs at runtime instead of “mysterious” later crashes.
- When enabled with a non-zero value, trades performance for stronger checking and configurable behavior on detected corruption.
- May be useful as a zero-recompile debugging aid on legacy glibc-based systems





Dynamic/runtime analysis

- Valgrind
- ASAN
- UBSAN



Valgrind

- **Zero code modification required** - Works directly with program binaries through dynamic instrumentation.
- **No need for recompilation.**
- **Trade-off** - Provides 100% user-space code coverage including system libraries, but programs run 20-100x slower during analysis.



Valgrind - a set of tools

A suite of instrumentation-based tools

- **Memcheck** - memory errors: leaks, buffer overruns, use-after-free, and uninitialized reads.
- **Helgrind** - threading bugs: thread error detector for race conditions, lock ordering issues, and API misuse.
- **Cachegrind/Callgrind** - cache profiling: cache and branch-prediction profiler that generates call graphs for performance analysis.
- **Massif** - heap profiling: "Where/when is my peak memory usage?"



Valgrind - Memcheck

- Memory leaks
- Use of uninitialized memory
- Invalid reads/writes
- Improper allocation/deallocation



Valgrind usage example

[day02/08_valgrind/main.c](#)



Valgrind usage example

[exercises/day02/ex02](#)

Remember Maze Solver? So we are back...



Valgrind usage example - deep dive - leak categories

[day02/08_valgrind/main.cpp](#)



Valgrind leak summary

==<PID>== LEAK SUMMARY:

==<PID>== definitely lost: 40 bytes in 1 blocks

==<PID>== indirectly lost: 0 bytes in 0 blocks

==<PID>== possibly lost: 0 bytes in 0 blocks

==<PID>== still reachable: 0 bytes in 0 blocks

==<PID>== suppressed: 0 bytes in 0 blocks



definitely lost

This memory was allocated on the heap but never freed, and the program no longer has any pointer to it.



indirectly lost

This memory was also never freed.

But it may seem still reachable through another block that is itself lost.

Typical example: a node in the middle of a linked list that is reachable only from a head node that has been leaked. Fixing the “definitely lost” block often fixes the “indirectly lost” ones



still reachable

This memory was not freed at program exit, but the program still held at least one valid pointer to it when it finished. In some cases this is intentional (e.g., global data structures destroyed at a shutdown phase), but it can also indicate that cleanup logic is missing.



possibly lost

Valgrind sees that the only pointer to unreleased heap memory, points somewhere inside a block, not at the start of the allocation. That usually means the pointer might be an offset, so Valgrind is unsure whether the memory is definitely lost (truly unreachable) or still reachable. In 99% of the cases it is an actual leak.



suppressed

Valgrind did not report any leaks from this category because they were filtered out by a suppression file. Suppressions are used to ignore known leaks (e.g., from third-party libraries or intentional allocations) that you don't want to see in the report.

Suppressing Errors: The Syntax

Location Syntax

Prefixes for specific types:

obj: Shared objects

fun: Functions

src: Source lines

Wildcards supported:

* and ?

Source line format:

filename[:lineNumber]

```
{  
  __gconv_transform_ascii_internal/mbtowc  
  Memcheck:Value4  
  fun:__gconv_transform_ascii_internal  
  fun:__mbr*toc  
  fun:mbtowc  
}
```

Exact Match Required

Every detail in the stack must correspond exactly to be suppressed.

Suppressing Errors: Use Mangled Names Only!

C++ function names must be **mangled** in suppressions!

Use the `--demangle=no` option to get mangled names in error messages.

IN SOURCE CODE

```
QListView::show()
```



MANGLED C++ NAME

```
_ZN9QListView4showEv
```

Suppressing Errors: Automatic Generation

The easiest way to add suppressions

```
--gen-suppressions=yes
```

Use this command-line option to automatically generate suppression specifications for errors that occur during execution.

Note: For best results, manual editing of the output is often advisable.



Suppressing Errors: **Suppression Files**

Default Configuration

Valgrind uses a standard suppression file by default:

```
$PREFIX/lib/valgrind/default.supp
```

Disable with:

```
--default-suppressions=no
```

Custom Suppressions

Specify your own suppression files using the command line flag:

```
--suppressions=/path/to/file.supp
```

Note: You can specify one or multiple files by repeating the flag.

Suppressing Errors: Report Suppression

If you use the `-v` option, Valgrind reports summary statistics for each used suppression at the end of execution.

EXAMPLE COMMAND

```
valgrind -v --tool=memcheck ls -l
```

OUTPUT SUMMARY

```
--1610-- used_suppression: 2 dl-hack3-cond-1 /usr/lib/valgrind/default.supp:1234  
--1610-- used_suppression: 2 glibc-2.5.x-on-SUSE-10.2-(PPC)-2a ...
```

Suppressions file - demo

[day02/08_valgrind/main.cpp](#)

Additional useful Valgrind flags (1)

`--trace-children=<yes|no> [default: no]`

When enabled, Valgrind will trace into sub-processes initiated via the `exec` system call. This is necessary for multi-process programs.

Additional useful Valgrind flags (2)

`--child-silent-after-fork=<yes|no> [default: no]`

When enabled, Valgrind will not show any debugging or logging output for the child process resulting from a fork call.

Additional useful Valgrind flags (3)

`--track-fds=<yes|no|all> [default: no]`

When enabled, Valgrind will print out a list of open file descriptors on exit or on request, via gdbserver, using: *monitor command v.info open_fds*.

Additional useful Valgrind flags (4)

--time-stamp=<yes|no> [default: no]

When enabled, each message is preceded with an indication of the elapsed wallclock time since startup.

--log-file=<filename>

Specifies that Valgrind should send all of its messages to the specified file. Use %p in <filename> for current process ID.

When debug symbols are stripped...

Valgrind looks for the .dwo/.dwp files in the executable directory.

Alternatively, use debuginfod with valgrind flag:

--enable-debuginfod=<no|yes> [default: yes]



Valgrind and GDB integration

Combine dynamic analysis with interactive debugging

- Valgrind detects memory errors during runtime
- When error occurs, program halts and GDB takes control
- GDB provides interactive inspection at the exact point of failure

Just launch valgrind with --vgdb flag and attach GDB when needed



Valgrind and GDB - the mechanism (1)

- **vgdb intermediary process**
 - Acts as relay between Valgrind's gdbserver and GDB client
- **Embedded gdbserver in Valgrind**
 - Valgrind runs the program with built-in GDB stub
 - Allows debugger control without separate gdbserver process



Valgrind and GDB - the mechanism (2)

- **Shared memory & FIFO communication**
 - vgdb uses FIFOs and shared memory to connect GDB commands to Valgrind's instrumented process without TCP sockets
- **Monitor command bridge**
 - GDB's monitor prefix routes Valgrind-specific commands through vgdb to Valgrind's command interpreter



Valgrind and GDB - a practical workflow

- **Launch with error trigger** - Start program:
`valgrind --vgdb=yes --vgdb-error=0 --vgdb-stop-at=startup <app>`
- **Attach GDB to running process** - `gdb <app>`, then `target remote | vgdb`
- **Debug at error point** - When Valgrind detects memory error, it halts; use GDB to inspect stack, variables, and exact failure location
- **Issue Valgrind commands from GDB** - Use `monitor leak_check`, `monitor v.info location <addr>` to query memory state while debugging

[Monitor commands reference](#)

Valgrind vgdb important flags (1)

`--vgdb=<no|yes|full> [default: yes]`

Valgrind will provide "gdbserver" functionality when `--vgdb=yes` or `--vgdb=full`

- Allows an external GNU GDB debugger to control and debug your program when it runs on Valgrind.
- `--vgdb=full` incurs significant performance overheads, but provides more precise breakpoints and watchpoints.

Valgrind vgdb important flags (2)

--vgdb-error=<number> [default: 999999999]

Wait for "number" errors to be reported before freezing the program and waiting for you to connect with GDB.

--vgdb-error=0 will cause the gdbserver to be started before your program is executed. This is typically used to insert GDB breakpoints before execution.

Valgrind vgdb important flags (3)

--vgdb-stop-at=<set> [default: none]

A comma separated list of one or more of:

- startup
- exit
- abexit
- valgrindabexit.

The values startup exit valgrindabexit respectively indicate to invoke gdbserver before your program is executed, after the last instruction of your program, on Valgrind abnormal exit.



VGDB example

[day02/08_valgrind](#) - memory issues

[day02/09_vgdb](#) - out of bound write (crash)



Address Sanitizer (ASAN)

- Fast compiler-based memory error detector for C++ (developed by Google)
- Instrumentation tool that combines compile-time transformation + runtime
- Built into GCC (≥ 4.8) and Clang
- Almost zero false positives (unlike Valgrind which can have false positives)



Errors that ASAN can detect

- Heap buffer overflow / underflow
- Stack buffer overflow / underflow
- Global variable buffer overflow / underflow
- Use-after-free (with exact allocation/deallocation sites)
- Double-free
- Alloc-dealloc mismatch
- Stack-use-after-return
- Container overflow
- Memory leaks (partial detection)



How ASAN works?

- Maps every 8 bytes of app memory → 1 byte of shadow memory
- Shadow byte = 0: address is valid ("unpoisoned")
- Shadow byte ≠ 0: address is invalid ("poisoned")
- Every memory access guarded with shadow memory check before execution
- Minimal overhead: ~2x slowdown average



ASAN Limitations

- Doesn't detect uninitialized variable reads (MemorySanitizer, clang)
- Requires recompilation (not a postmortem tool)
- May require -O0 or -Og (optimization may hide issues)
- Runtime overhead unacceptable for production



ASAN usage example

[day02/10_asan](#)



ASAN doesn't catch all issues

Try this:

<https://godbolt.org/z/9rxfrdYax>

And now this

<https://godbolt.org/z/Pq3o3afeT>

ASAN vs. Valgrind - features comparison

Feature	AddressSanitizer (ASAN)	Valgrind (Memcheck)
Speed	2-4x slowdown	20-50x slowdown
Memory Use	2-3x	10x+
Stack Overflow	✓ Caught	✗ Missed
Heap Overflow	✓ Caught	✓ Caught
Uninitialized	✗ (use MSAN)	✓ Caught
Multi-thread	✓ Excellent	✗ Poor
Recompilation	Required	Not required



ASAN vs. Valgrind - when to use

Can you recompile?

- No - Valgrind
- Yes - Try ASAN first

Did ASAN find the issue?

- No - Try Valgrind
- Yes - Good!

Is it life-critical system?

- Use both!

What's your runtime overhead tolerance?

- Low (~2x slowdown) - ASAN
- High (2-30x slowdown) - Both



UBSAN

- **What is UBSan?** Compiler-based sanitizer detecting undefined behavior at runtime
- **Language scope:** C, C++, Objective-C, Swift
- **Compilation:** `-fsanitize=undefined` flag instruments code for UB checks (no binary instrumentation like ASAN, only source-level)
- **Overhead:** Minimal (~5-10% slowdown typical) compared to ASAN's 2x overhead - suitable for production debugging



What UBSAN checks for?

Integer checks: Signed overflow, unsigned wrap, shifts by count $\geq$ bit-width

Pointer checks: Null dereference, unaligned access, derived-to-base cast errors, member access on invalid pointers

Type checks: Invalid type casts, vptr dereferencing on null/invalid objects, load from uninitialized memory

Bounds checks: Array subscript out-of-bounds on stack/static arrays

Logic violations: Unreachable code, enum value outside range, invalid bool construction



Compiler flags for selective checking

Compiler flags	Explanation
<code>-fsanitize=undefined</code> <code>-fno-sanitize=signed-integer-overflow</code>	Turn on all, turn off specific Exclude expensive checks (or intended behavior)
<code>-fsanitize=null,divmod,bool,return</code>	Lightweight subset for production
<code>-fsanitize-recover=undefined</code>	Continue execution after UB detection (log only)
<code>-fsanitize=implicit-conversion</code>	Detects lossy casts (float→int, signed→unsigned)
<code>__attribute__((no_sanitize("undefined")))</code>	Suppress checks in specific files/functions



Practical workflow

- **Development:** Full UBSan with all checks enabled (`-fsanitize=undefined`)
- **Testing:** `-fsanitize=undefined -fsanitize-recover=undefined` with log output for CI/CD pipelines
- **Production debugging:** Lightweight subset (`-fsanitize=null,divmod`) when field crashes need investigation
- **Output control:** `UBSAN_OPTIONS=print_stacktrace=1:halt_on_error=0` - fine-tune verbosity and behavior (environment variable)



UBSAN example

[day02/13_ubsan](#)



Heap memory structure

Higher Addresses ↑

```
+-----+ <--- Top Chunk (The "Wilderness")
|      Free Chunk      | Available for new allocations
+-----+
|      Allocated Chunk  | Current user data
+-----+
|      Free Chunk      | Linked to a Bin (Small/Fast/etc.)
+-----+
|      Allocated Chunk  |
+-----+ <--- main_arena / Heap Start
```

Lower Addresses



Process memory structure

[text][data][bss][heap->] [mmap] ... [<- stack]

Heap





Heap memory structure: Chunks

The heap is made of chunks - blocks of memory managed by the allocator

User program gets a pointer to the usable part of the chunk

The allocator keeps some hidden information about the chunk (free/in use, size, ...)

```
[ chunk header ][ your bytes here ]  
                ^  
                |  
            pointer returned by malloc
```



The bins (Recycling Lists)

When memory is freed, the allocator rarely returns it directly to the OS. Instead, it caches these 'chunks' in optimized recycling lists (bins) for fast reuse in future malloc calls.

Those remembered lists are called bins. In simple terms:

- **Fastbins:** very small freed chunks kept for ultra-/quick reuse.
- **Unsorted bin:** a temporary waiting area for newly freed chunks before they get sorted into the right bin
- **Small & Large bis:** larger freed chunks grouped by size.



Wilderness / Top Chunk

The Top Chunk is the continuous block of unallocated memory at the very top of the heap. It is not stored in a bin; malloc uses it as a last resort, carving off a piece only when no suitable recycled chunks are available in the bins.

```
[used][used][free in bin][used][ TOP CHUNK ..... ]
```



The main arena

main_arena is the primary bookkeeping structure for the process heap. It is a global struct where glibc stores the "state" of the allocator, including all bins, a pointer to the Top Chunk, and synchronization locks to ensure thread safety.



Where freed chunks go

fastbins:

Small freed chunks can stay in fastbins for very fast reuse. Single linked list. Not immediately consolidated.

unsorted → bins:

Larger freed chunks commonly visit the unsorted bin first, then move into small or large bins as allocation logic reorganizes them.

top fallback:

If no free chunk fits, malloc slices space from the top chunk; the remainder becomes the new top.



Heap inspection using GDB built-in tools

- Run `info proc mappings` to identify heap region (look for `[heap]` label)
- Examine `main_arena`: `print main_arena` for glibc's heap control structure
- Print bins: `print main_arena.bins` for showing all bins
- Print fastbins: `p main_arena.fastbinsY` (can be indexed from 0 to 9)



GDB built-in heap inspection - example

[day02/11_inspect_heap](#)



Analyzing heap memory with Python script

GDB can be extended with Python scripts to analyze and display glibc heap information, such as chunks, bins, and arenas, beyond vanilla commands like `x/` for memory dumps.



Popular extensions

- **GEF**: Adds `heap`, `heap bins`, `visualize-libc-heap-chunks` for glibc arenas, fastbins, and chunk layouts (<https://hugsy.github.io/gef/>)
- **Pwndbg**: Features `heap` to print chunks, bins (fast, unsorted, small/large), and tcache, ideal for exploit debugging(<https://github.com/pwndbg/pwndbg>)
- **gdb-heap**: Python script with `heap used/free/all/sizes` for usage reports, diffs, and categorization (<https://github.com/rogerhu/gdb-heap>)



`gdb-heap`

For comprehensive heap analysis, we'll use the **`gdb-heap`** Python module

Installation:

- **`git clone https://github.com/rogerhu/gdb-heap`**
- Import the **`gdbheap`** Python module
`(gdb) python import gdbheap`



gdb-heap

After installation, the following commands are available:

Command	Description	Command	Description
heap	Memory usage by category	heap arenas	Show glibc arenas
heap sizes	Memory usage by allocation size	heap sizes	List all sizes
heap free	Show free chunks	heap all	All chunks
heap used	Show used chunks	hexdump 0xADDR	



gdb-heap demo

[day02/12_gdb_heap](#)



Session 4 summary

- Tracepoints - debt
- Debugging memory issues with GDB
- Runtime memory analysis with Valgrind + Valgrind - GDB integration
- ASAN and UBSAN
- Heap inspection



What's next?

- Multithreading
- GDB customization